

# numba-cuda vs numba-cuda-mlir

**The question:** does the choice of CUDA compiler backend change performance — and where? **The method:** profile the identical Stable-Fluids GPU solver (one source file, 9 kernels) built by both compilers, holding the GPU, CUDA 13.3 toolchain, workload, and Python version constant — so any difference is the compiler alone.

**2**

COMPILERS · ONE IDENTICAL SOURCE

**9**KERNELS · GRIDS  $256^2$ – $2048^2$ **5**

AXES: JIT · CODE · RUNTIME · LAUNCH · GRAPHS

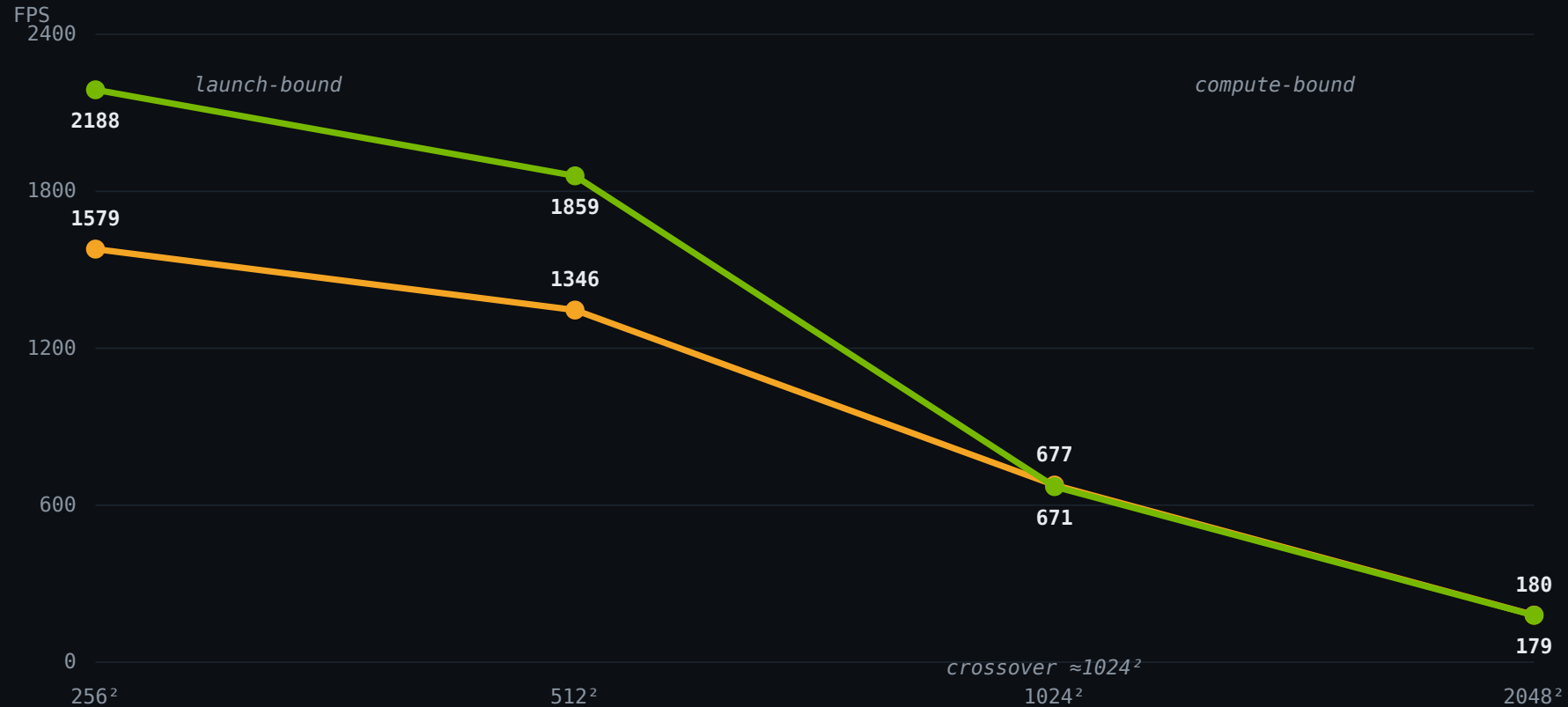
# One variable: the compiler frontend

|                       | CLASSIC                                | MLIR                  |
|-----------------------|----------------------------------------|-----------------------|
| package               | numba-cuda 0.30.2                      | numba-cuda-mlir 0.4.0 |
| frontend              | Numba IR → LLVM → NVVM → PTX           | MLIR → PTX (LTOIR)    |
| opt / LTO default     | NVVM opt-3, no LTO                     | LTO, opt>0            |
| Python                | 3.12.13                                | 3.12.13               |
| cuda-bindings / ptxas | 13.3.1 / CUDA 13.3                     | 13.3.1 / CUDA 13.3    |
| GPU                   | RTX 5880 Ada (sm_89), driver 595.71.05 |                       |

Bit-exact output on all 9 kernels at  $256^2$  /  $512^2$  /  $1024^2$  /  $2048^2$  → a fair race on identical work.

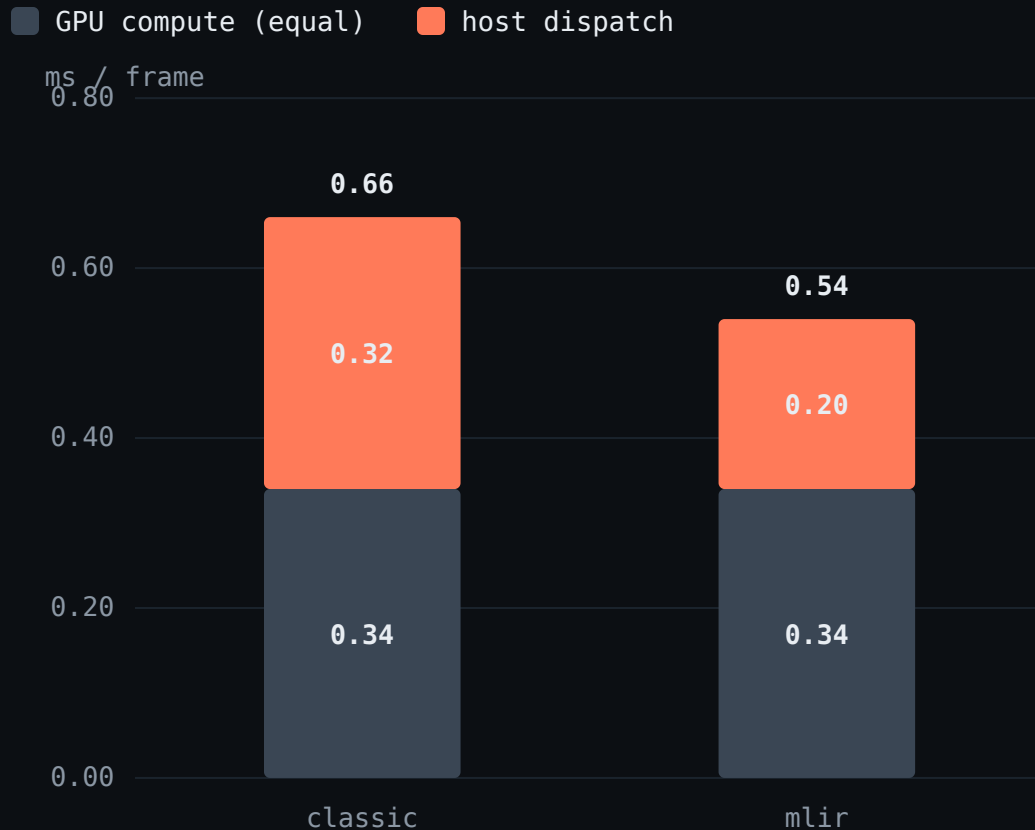
# The launch-bound → compute-bound crossover

■ classic (NVVM)   ■ mlir (MLIR)



Small grids = **host-launch-bound** (37 launches/frame): mlir wins **~38%**. Big grids = GPU-bound: **identical**.

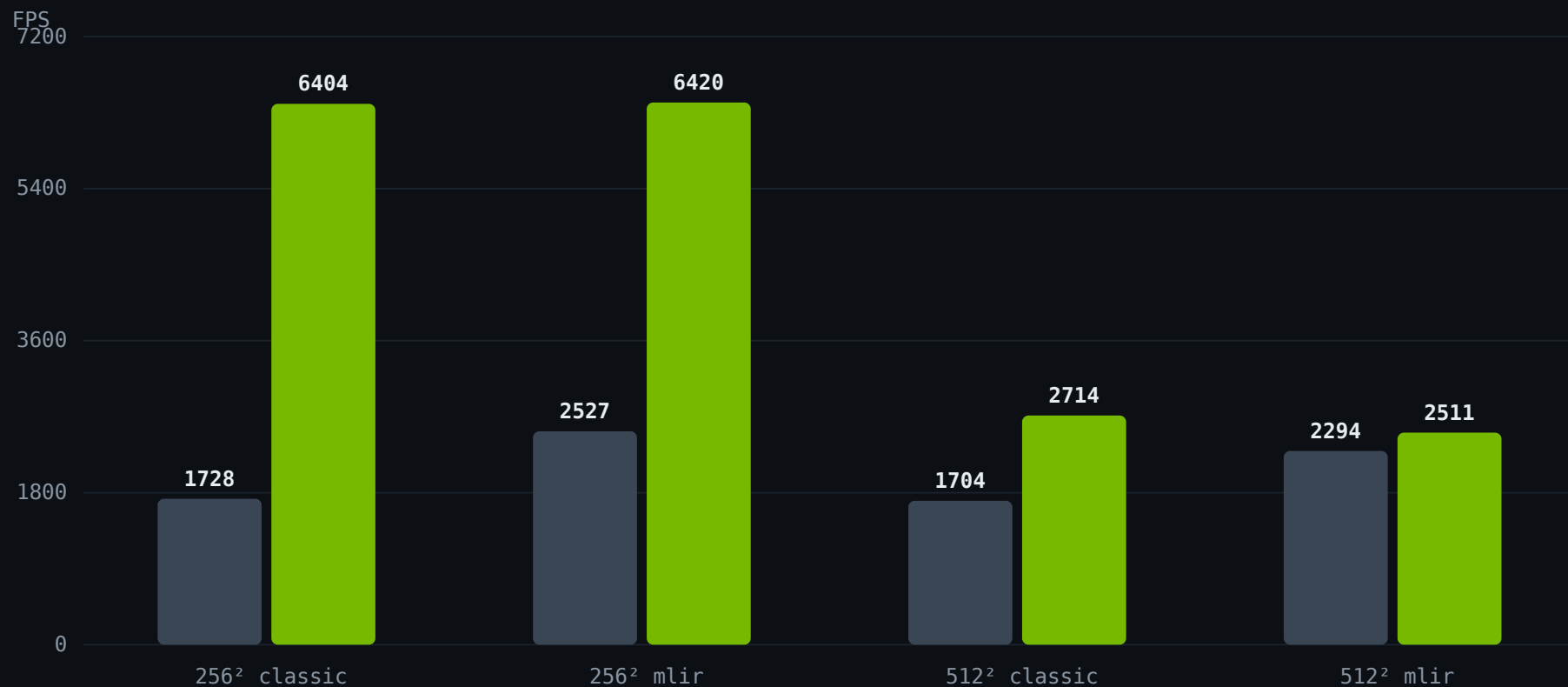
# Equal GPU time; unequal dispatch cost



- ▶ The steel base — GPU compute — is the **same 0.34 ms** in both.
- ▶ The coral top — host dispatch — differs: **8.6  $\mu$ s** vs **5.3  $\mu$ s** per launch.
- ▶  $\times 37$  launches/frame builds the total; mlir is **~38% lighter** to dispatch.
- ▶ **Why:** classic runs  $\sim 2\times$  the Python per launch (**69** vs **34** calls) — re-resolving the CUDA context and re-prepping args every time; mlir caches both.

# CUDA graphs erase the gap

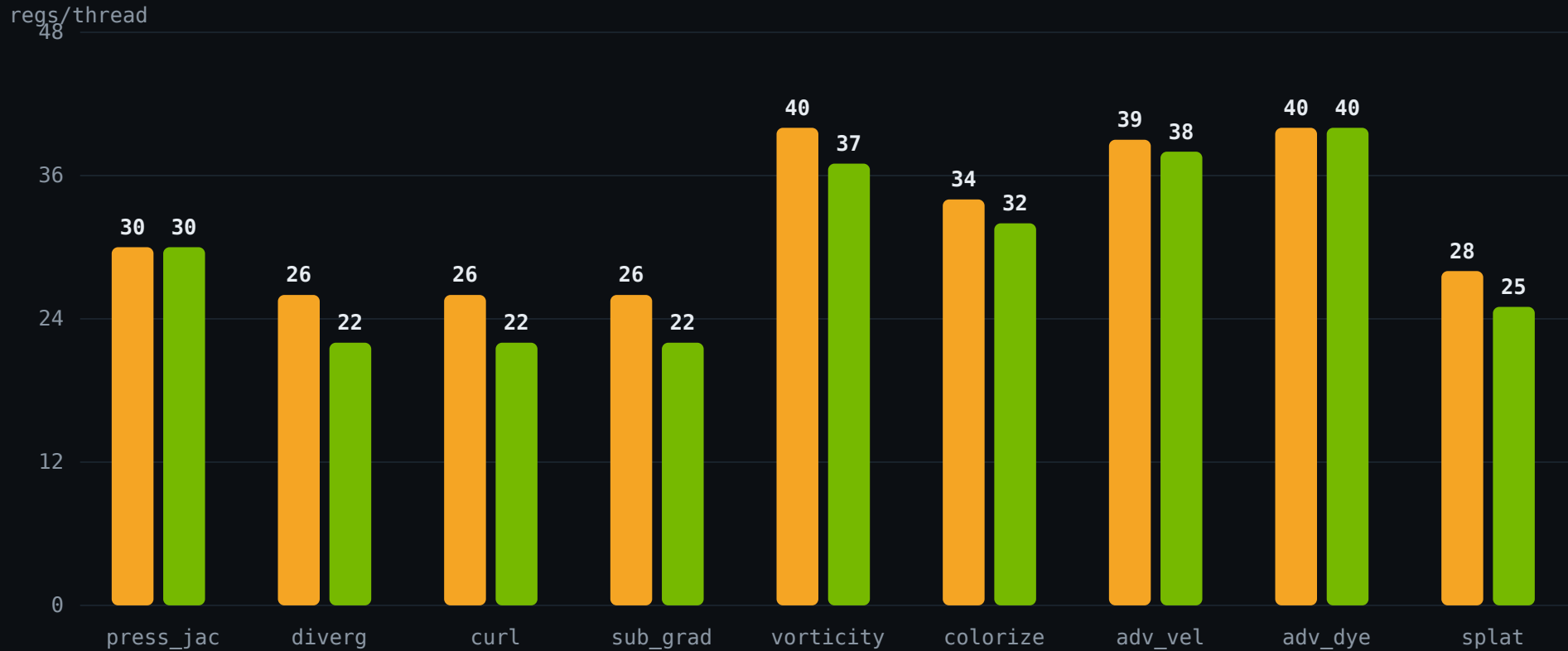
■ eager ■ graphed



Capture once, replay as a single launch. Both **converge to the same FPS** – classic gains more (**3.7×** vs 2.5×). mlir's eager edge **disappears**.

# Slightly fewer registers — and it doesn't matter

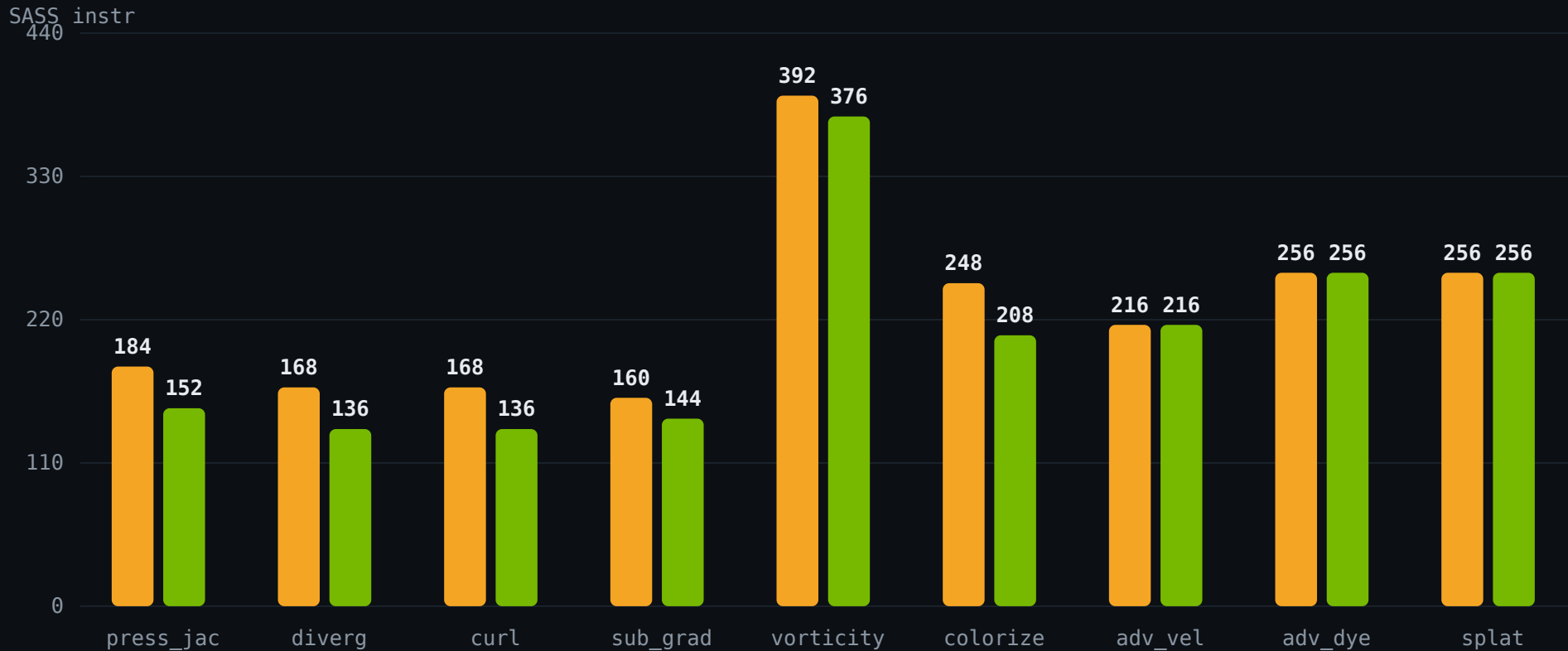
classic mlir



All below the 42-reg occupancy limit → no occupancy change. Dominant `pressure_jacobi`: 30 = 30.

# Fewer instructions $\neq$ faster here

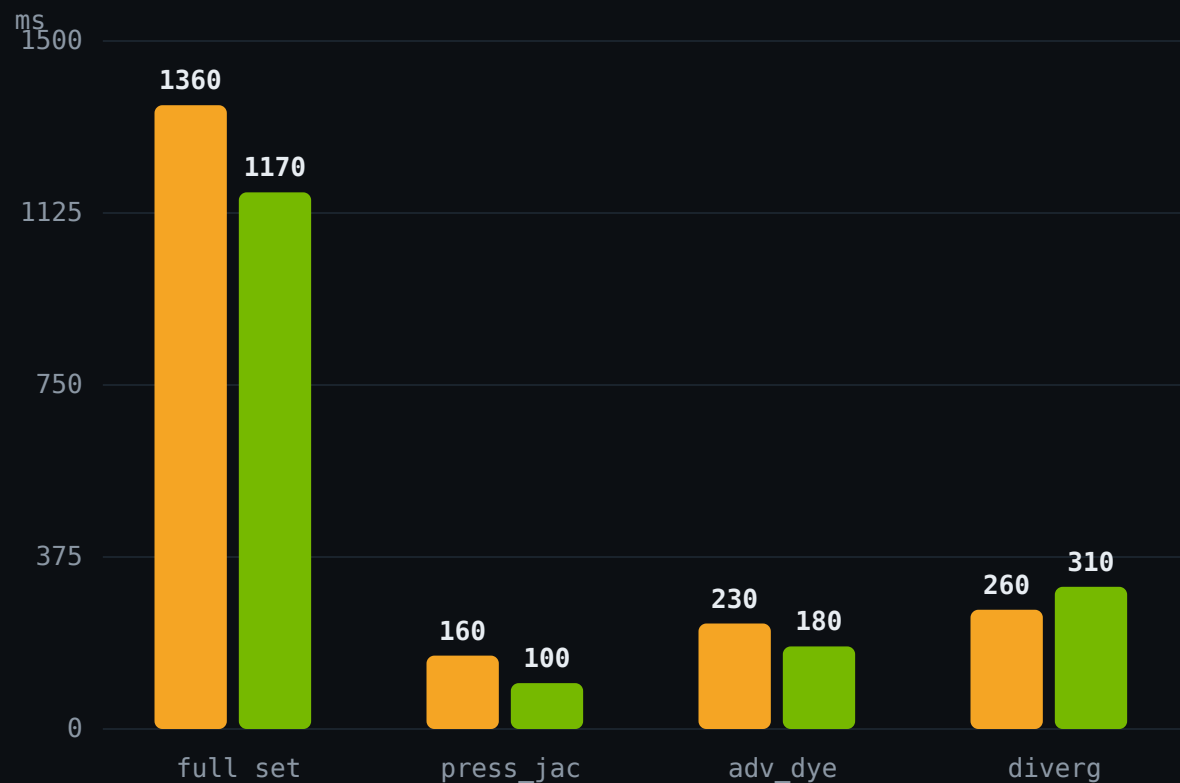
classic mlir



mlir trims -10-19% on stencils, classic wins colorize – yet GPU time is identical. The limiter is **float64 throughput** (Python literals promote), not instruction count.

# mlir compiles ~14% faster

■ classic ■ mlir



**1.36s**

CLASSIC – FULL SET

**1.17s**

MLIR – FULL SET (3.12)

Context pre-warmed → pure compile. No on-disk cache, so paid every process.



# Match the settings, and the compilers tie

| REGISTERS / THREAD<br>pressure_jacobi · fewer = better | LTO OFF | LTO ON |
|--------------------------------------------------------|---------|--------|
| classic                                                | 30      | 30     |
| mlir                                                   | 50      | 30     |

- ▶ **LTO** (link-time optimization) is a knob **both** compilers have.
- ▶ mlir turns it **on** by default; classic leaves it **off** — that's the whole reason mlir's code looked tighter on slides 5–6.
- ▶ Turn LTO on for both → **identical, 30 regs**. Neither compiler is fundamentally better.

# A tie on everything — except how fast they launch kernels

| WHAT WE MEASURED               | WINNER | BY HOW MUCH                                                            |
|--------------------------------|--------|------------------------------------------------------------------------|
| output correctness             | tie    | exact same numbers, every grid size                                    |
| GPU kernel speed               | tie    | same runtime & occupancy                                               |
| GPU code size                  | tie*   | mlir writes ~15% fewer instrs – but no faster; equal once both use LTO |
| compile time                   | mlir   | ~14% faster                                                            |
| kernel-launch overhead         | mlir   | 5.3 vs 8.6 $\mu$ s/launch $\rightarrow$ +38% FPS on small grids        |
| FPS on big grids / with graphs | tie    | identical                                                              |

**Core idea:** for performance the two compilers are interchangeable – same results, same kernel speed. mlir's **only** real edge is cheaper kernel launching, which matters on small workloads and **vanishes with CUDA graphs**.

# Pick on launch model, not the compiler

- ▶ **Same results, same kernel speed** — identical output; equal GPU time and occupancy.
- ▶ **mlir launches kernels ~38% cheaper** — 5.3 vs 8.6  $\mu\text{s}$   $\rightarrow$  +38% FPS below 1024<sup>2</sup>.
- ▶ **CUDA graphs  $\approx 3\times$  FPS** on both backends — and erase the mlir-vs-classic gap.
- ▶ **mlir compiles ~14% faster** — 1.17 s vs 1.36 s for all 9 kernels.
- ▶ **Biggest latent win:** kernels run in float64 (Python literals); float32 would cut math on both.